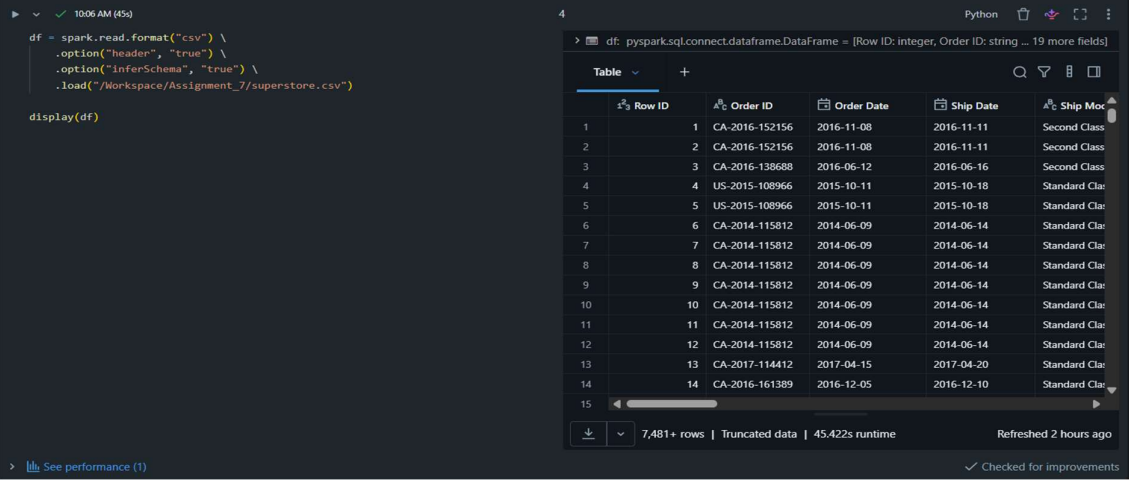


Screenshots(Output)

Loading superstore Dataset



The screenshot shows a Jupyter Notebook interface. On the left, the code cell contains the following Python code:

```
df = spark.read.format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .load("/Workspace/Assignment_7/superstore.csv")

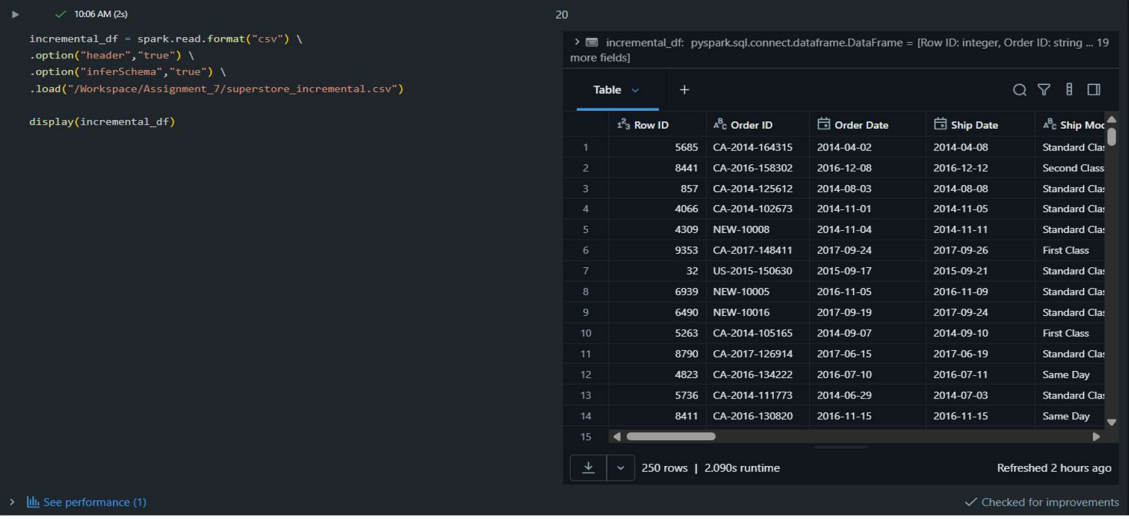
display(df)
```

On the right, the table view displays the first 15 rows of the dataset. The table has columns: Row ID, Order ID, Order Date, Ship Date, and Ship Mode. The data is truncated, showing only 15 rows out of 7,481 total rows.

Row ID	Order ID	Order Date	Ship Date	Ship Mode
1	CA-2016-152156	2016-11-08	2016-11-11	Second Class
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class
3	CA-2016-138688	2016-06-12	2016-06-16	Second Class
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class
5	US-2015-108966	2015-10-11	2015-10-18	Standard Class
6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class
7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class
8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class
9	CA-2014-115812	2014-06-09	2014-06-14	Standard Class
10	CA-2014-115812	2014-06-09	2014-06-14	Standard Class
11	CA-2014-115812	2014-06-09	2014-06-14	Standard Class
12	CA-2014-115812	2014-06-09	2014-06-14	Standard Class
13	CA-2017-114412	2017-04-15	2017-04-20	Standard Class
14	CA-2016-161389	2016-12-05	2016-12-10	Standard Class

At the bottom of the table view, it indicates 7,481+ rows, truncated data, and a runtime of 45.422s. The table was refreshed 2 hours ago.

Loading superstore_incremental Dataset



The screenshot shows a Jupyter Notebook interface. On the left, the code cell contains the following Python code:

```
incremental_df = spark.read.format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .load("/Workspace/Assignment_7/superstore_incremental.csv")

display(incremental_df)
```

On the right, the table view displays the first 15 rows of the incremental dataset. The table has columns: Row ID, Order ID, Order Date, Ship Date, and Ship Mode. The data is truncated, showing only 15 rows out of 250 total rows.

Row ID	Order ID	Order Date	Ship Date	Ship Mode
1	5685 CA-2014-164315	2014-04-02	2014-04-08	Standard Class
2	8441 CA-2016-158302	2016-12-08	2016-12-12	Second Class
3	857 CA-2014-125612	2014-08-03	2014-08-08	Standard Class
4	4066 CA-2014-102673	2014-11-01	2014-11-05	Standard Class
5	4309 NEW-10008	2014-11-04	2014-11-11	Standard Class
6	9353 CA-2017-148411	2017-09-24	2017-09-26	First Class
7	32 US-2015-150630	2015-09-17	2015-09-21	Standard Class
8	6939 NEW-10005	2016-11-05	2016-11-09	Standard Class
9	6490 NEW-10016	2017-09-19	2017-09-24	Standard Class
10	5263 CA-2014-105165	2014-09-07	2014-09-10	First Class
11	8790 CA-2017-126914	2017-06-15	2017-06-19	Standard Class
12	4823 CA-2016-134222	2016-07-10	2016-07-11	Same Day
13	5736 CA-2014-111773	2014-06-29	2014-07-03	Standard Class
14	8411 CA-2016-130820	2016-11-15	2016-11-15	Same Day

At the bottom of the table view, it indicates 250 rows and a runtime of 2.090s. The table was refreshed 2 hours ago.

Data Cleaning superstore Dataset

```
12 clean_df = df.dropna()
13 clean_df = clean_df.dropDuplicates()
14 print("Rows after cleaning :", clean_df.count())
display(clean_df)
15 clean_df = clean_df.dropDuplicates(["Order_ID", "Product_ID"])
```

Rows after cleaning : 9994

Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	
1	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335
2	17	CA-2014-105893	2014-11-11	2014-11-18	Standard Class	PK-19075
3	21	CA-2014-143336	2014-08-27	2014-09-01	Second Class	ZD-21925
4	53	CA-2015-115742	2015-04-18	2015-04-22	Standard Class	DR-13000
5	118	CA-2015-110457	2015-03-02	2015-03-06	Standard Class	DK-12090
6	140	CA-2016-145263	2016-10-13	2016-10-19	Standard Class	LC-16885
7	145	CA-2017-155376	2017-12-22	2017-12-27	Standard Class	SG-20080
8	147	CA-2014-110072	2014-10-22	2014-10-28	Standard Class	MG-17680
9	186	CA-2016-105018	2016-11-28	2016-12-02	Standard Class	SK-19990
10	200	US-2017-124303	2017-07-06	2017-07-13	Standard Class	FH-14365
11	237	CA-2017-160514	2017-11-12	2017-11-16	Standard Class	DB-13120
12	246	CA-2014-131606	2014-06-01	2014-06-06	Second Class	DW-13480
13	260	CA-2017-163139	2017-12-01	2017-12-03	Second Class	CO-12670
14	263	US-2014-109992	2014-09-19	2014-09-21	Second Class	SB-20290

7,485+ rows | Truncated data | 5.457s runtime | Refreshed 2 hours ago

Data Cleaning superstore_incremental Dataset

```
27 incremental_df = incremental_df.dropna()
28 incremental_df = incremental_df.dropDuplicates()
29 incremental_df = incremental_df.dropDuplicates(["Order_ID", "Product_ID"])
30 print("Rows after cleaning :", incremental_df.count())
```

Rows after cleaning : 258

✓ Checked for improvements

Creating superstore_delta Delta Table

```
18 table_name = "superstore_delta"
clean_df.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable(table_name)
```

Optimize

Creating superstore_incremental Delta Table

```
34 table_name2 = "superstore_incremental"
incremental_df.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable(table_name2)
```

Optimize

Applying Merge

```
35 main_table = DeltaTable.forName(spark, "superstore_delta")
incremental_table = DeltaTable.forName(spark, "superstore_incremental")
37 merge_result = main_table.alias("target") \
    .merge(
        incremental_table.toDF().alias("source"),
        """
        target.Order_ID = source.Order_ID
        AND
        target.Product_ID = source.Product_ID
        """) \
    .whenMatchedUpdateAll() \
    .whenNotMatchedInsertAll() \
    .execute()
```

Optimize

Validating Results

10:06 AM (14)

```
print("Total Records after MERGE:", final_df.count())
```

See performance (1)

Total Records after MERGE: 10036

Checked for improvements

10:06 AM (14)

```
duplicates = final_df.groupBy("Order_ID", "Product_ID").count().filter("count > 1")
display(duplicates)
```

See performance (1)

duplicates: pyspark.sql.connect.dataframe.DataFrame = [Order_ID: string, Product_ID: string ... 1 more field]

Table

Order_IDProduct_IDcount

No rows returned

0 rows | 1.491s runtime

Refreshed 2 hours ago

Optimize

Displaying Final Result

10:06 AM (14)

```
final_df = spark.table("superstore_delta")
display(final_df)
```

See performance (1)

final_df: pyspark.sql.connect.dataframe.DataFrame = [Row_ID: integer, Order_ID: string ... 19 more fields]

Table

Row_IDOrder_IDOrder_DateShip_DateShip_ModeCustomer_ID

1	8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710
2	24	US-2017-156909	2017-07-16	2017-07-18	Second Class	SF-20065
3	50	CA-2015-115742	2015-04-18	2015-04-22	Standard Class	DP-13000
4	58	CA-2016-111682	2016-06-17	2016-06-18	First Class	TB-21055
5	59	CA-2016-111682	2016-06-17	2016-06-18	First Class	TB-21055
6	106	US-2015-156867	2015-11-13	2015-11-17	Standard Class	LC-16870
7	113	CA-2016-128867	2016-11-03	2016-11-10	Standard Class	CL-12565
8	114	CA-2014-115259	2014-08-25	2014-08-27	Second Class	RC-19960
9	115	CA-2014-115259	2014-08-25	2014-08-27	Second Class	RC-19960
10	117	CA-2014-115259	2014-08-25	2014-08-27	Second Class	RC-19960
11	118	CA-2015-110457	2015-03-02	2015-03-06	Standard Class	DK-13090
12	129	US-2016-125969	2016-11-06	2016-11-10	Second Class	LS-16975
13	131	US-2017-164147	2017-02-02	2017-02-05	First Class	DW-13585
14	208	CA-2017-135860	2017-12-01	2017-12-07	Standard Class	JH-15985
15						

7,485+ rows | Truncated data | 2.756s runtime

Refreshed 2 hours ago

Checked for improvements